

# End-to-End Data Pipeline for Semantic Search

## Project Report

### Team Members:

Andrei-Daniel Anghelescu  
Rares-Alexandru Constantin  
Robert Granca  
Ana-Maria Toader

Faculty of Automatic Control and Computers  
University POLITEHNICA of Bucharest

January 2026

# Contents

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                       | <b>2</b>  |
| <b>2</b> | <b>Individual Contributions</b>                                           | <b>4</b>  |
| 2.1      | Building a Resilient Data Ingestion Layer with Apache Kafka . . . . .     | 4         |
| 2.2      | Developing a Scalable Web Scraper for Data Aggregation . . . . .          | 6         |
| 2.3      | Orchestrating Services and Visualizing Data with OpenSearch . . . . .     | 8         |
| 2.4      | Implementing a Semantic Processing Pipeline for Text Embeddings . . . . . | 10        |
| <b>3</b> | <b>Implementation Description</b>                                         | <b>12</b> |
| <b>4</b> | <b>Conclusions</b>                                                        | <b>13</b> |

## Abstract

This project presents the design and implementation of a scalable end-to-end Big Data pipeline for semantic search, addressing the growing need for intelligent information retrieval systems that go beyond traditional keyword matching. The system integrates multiple cutting-edge technologies to create a robust pipeline capable of acquiring web data, processing it through message queues, generating vector embeddings, and indexing it into OpenSearch for hybrid (lexical + semantic) search retrieval.

The architecture follows the Extract, Transform, Load (ETL) pattern enhanced for vector search capabilities. Python-based web scrapers handle data acquisition by fetching HTML content, cleaning it, and extracting raw text. Apache Kafka serves as the message queue backbone, buffering raw documents and ensuring reliable data flow between components. The transformation layer employs sentence-transformers to generate 384-dimensional vector representations of text chunks created through a sliding window approach with configurable overlap. OpenSearch provides the storage and retrieval layer, supporting k-Nearest Neighbors (k-NN) queries for semantic similarity search alongside traditional full-text search.

The system is fully containerized using Docker Compose, enabling consistent deployment across environments and simplified service orchestration. A Streamlit-based frontend provides an intuitive interface for querying the indexed documents using semantic, text-based, or hybrid search modes. This project demonstrates the practical application of modern data engineering principles to build an intelligent search system suitable for enterprise knowledge management, content discovery, and information retrieval applications.

## 1 Introduction

The exponential growth of digital content has created unprecedented challenges in information retrieval. Traditional search engines, while effective for keyword-based queries, often fail to capture the semantic intent behind user searches. A query for "how to reduce development time" might miss highly relevant documents discussing "accelerating software delivery" or "streamlining the coding process" simply because they lack exact keyword matches. This semantic gap has driven the development of vector-based search systems that understand meaning rather than just matching text patterns.

Recent advances in natural language processing, particularly in transformer-based models and sentence embeddings, have made semantic search both practical and scalable. Models like sentence-transformers can encode text into dense vector representations that capture semantic meaning, allowing similar concepts to cluster together in high-dimensional space. When combined with efficient approximate nearest neighbor algorithms, these embeddings enable real-time semantic similarity search across millions of documents. However, building production-ready semantic search systems requires solving numerous engineering challenges: reliable data ingestion, efficient text processing, vector storage and indexing, and seamless integration of multiple services.

This project addresses these challenges by constructing a complete data pipeline that brings together best-in-class open-source technologies. Apache Kafka provides the messaging backbone for reliable, scalable data ingestion with built-in fault tolerance. OpenSearch delivers enterprise-grade search capabilities with native support for vector indexing through its k-NN plugin. Python serves as the glue connecting these systems,

handling web scraping, text chunking, embedding generation, and orchestration. By containerizing all components with Docker Compose, the system achieves portability and reproducibility while maintaining the flexibility to scale individual services as needed. The result is a practical, extensible foundation for building intelligent search applications that understand both keywords and meaning.

## 2 Individual Contributions

### 2.1 Building a Resilient Data Ingestion Layer with Apache Kafka

In today's data-driven landscape, the ability to reliably ingest, buffer, and process large volumes of information is fundamental to any successful data pipeline. Apache Kafka has emerged as the de facto standard for distributed event streaming, and through this project, I have gained firsthand experience in leveraging its capabilities to build a resilient ingestion layer for our semantic search system.

#### The Case for Message Queues in Data Pipelines

Traditional point-to-point data architectures, where producers directly communicate with consumers, suffer from tight coupling and poor fault tolerance. When the downstream processing system fails or slows down, the entire pipeline grinds to a halt. Kafka addresses this by introducing an intermediate layer that decouples data producers from consumers. In our pipeline, the web scraper (producer) can continue fetching and streaming data regardless of whether the semantic processing consumer is operational. This decoupling proved invaluable during development, allowing us to independently test and optimize each component.

Kafka's distributed nature provides horizontal scalability that traditional databases struggle to match. A single Kafka topic can be partitioned across multiple brokers, with each partition handling a subset of the data. For our **raw-web-data** topic, this means we could theoretically scale scraping operations by adding more producers and partitions without architectural changes. The consumer group model further enhances this, multiple consumer instances can cooperatively process partitions, providing automatic load balancing and failover.

#### Implementing Resilient Producers and Consumers

The producer implementation required careful consideration of message delivery semantics. Kafka offers three acknowledgment modes: **acks=0** (fire-and-forget), **acks=1** (leader acknowledgment), and **acks=all** (full replica acknowledgment). For a data pipeline where losing documents is unacceptable, I chose **acks=all** to ensure every message is durably written before proceeding. Combined with **retries=3** and exponential backoff, our producer can weather temporary broker failures without data loss.

The consumer side presented its own challenges, particularly around offset management. Kafka tracks consumption progress through offsets, sequential numbers assigned to each message. By disabling automatic offset commits (**enable\_auto\_commit=False**), I implemented manual commit logic that only advances the offset after successful processing. This prevents scenarios where a crash mid-processing would cause messages to be silently skipped. The implementation uses synchronous commits after each batch, trading some throughput for stronger delivery guarantees.

#### Monitoring and Observability

A production Kafka deployment demands comprehensive monitoring. The wrapper classes I developed include statistics tracking for messages sent/received, bytes processed, and processing times. These metrics integrate with our logging framework (Loguru), providing

visibility into pipeline health. During testing, these statistics helped identify bottlenecks, we discovered that JSON serialization was consuming significant CPU time, leading us to implement batch processing optimizations.

### **Lessons Learned and Future Directions**

Working with Kafka reinforced several key principles. First, distributed systems require defensive programming, every network call can fail, and code must handle failures gracefully. Second, the importance of idempotent processing became clear; since Kafka provides at-least-once delivery, consumers must handle duplicate messages without corrupting state. Third, schema evolution is a real concern, changing the structure of messages requires careful coordination between producers and consumers.

Looking forward, I would explore Kafka Schema Registry for formal schema management and consider implementing exactly-once semantics using Kafka transactions for critical data paths. The experience of building this ingestion layer has provided a solid foundation for tackling larger-scale streaming challenges.

## 2.2 Developing a Scalable Web Scraper for Data Aggregation

Web scraping stands at the frontier of data acquisition, transforming the vast, unstructured expanse of the internet into structured data suitable for analysis and indexing. Building a scalable, respectful web scraper for our semantic search pipeline taught me valuable lessons about responsible data collection, HTTP mechanics, and the balance between speed and politeness.

### The Art and Ethics of Web Scraping

Before diving into implementation, it's essential to acknowledge the ethical considerations surrounding web scraping. Our scraper respects `robots.txt` directives, implements rate limiting to avoid overwhelming target servers, and focuses on publicly available educational content. The `delay_seconds` configuration ensures we never hammer a single domain, and user agent rotation helps distribute load while accurately identifying our crawler. These aren't just technical features, they reflect a commitment to being good citizens of the web.

### Architecture and Design Decisions

The scraper follows an object-oriented design centered on the `WebScraper` class and `ScrapedDocument` dataclass. Each scraped page produces a document containing the URL, extracted title, cleaned text content, timestamp, domain, and a unique document ID (generated via MD5 hash of the URL). This structure provides everything downstream components need while maintaining provenance information.

Request handling leverages the `requests` library with session management for connection pooling and cookie persistence. The `fake_useragent` library provides realistic user agent strings, preventing blocks from sites that reject obvious bot traffic. Each request includes proper headers mimicking browser behavior, Accept headers, referrer handling, and encoding preferences.

### Content Extraction Challenges

Converting HTML to clean text proved surprisingly complex. Raw HTML contains navigation menus, advertisements, footers, and other "boilerplate" content that pollutes extracted text. Our `ContentExtractor` class (implemented in a separate module) tackles this through a multi-stage approach: first removing script, style, and navigation elements; then applying readability heuristics to identify main content areas; finally cleaning whitespace and normalizing text.

BeautifulSoup provides the parsing foundation, with careful handling of character encodings that vary wildly across the web. We encountered pages mixing UTF-8 with legacy encodings, pages missing encoding declarations, and even pages with conflicting charset specifications in HTTP headers versus meta tags. Robust error handling and encoding detection libraries helped navigate this minefield.

### Handling the Unexpected

Real-world web scraping requires handling countless edge cases. Some pages block based on IP reputation; others require JavaScript rendering. Dynamic content loaded via AJAX remains invisible to simple HTTP requests. While our scraper focuses on static content

(appropriate for our educational data sources), I implemented hooks for future extension to browser-based scraping using tools like Playwright or Selenium.

Rate limiting demanded more sophistication than simple sleep calls. We track request timing per domain, ensuring we don't exceed our configured delay even when scraping multiple URLs from the same site in sequence. This domain-aware throttling prevents accidental denial-of-service scenarios while maximizing throughput across different domains.

## Integration with the Pipeline

The scraper integrates seamlessly with Kafka through the producer wrapper. Each scraped document is serialized to JSON and pushed to the `raw-web-data` topic. The producer handles batch accumulation, compression (gzip), and reliable delivery. This streaming architecture means downstream processing can begin as soon as documents arrive, there's no need to wait for scraping to complete.

Error handling follows a "fail-safe" philosophy. Individual URL failures are logged but don't halt the scraper. Network timeouts, HTTP errors, and parsing exceptions are caught, recorded, and skipped. The scraper maintains progress metrics showing success rates, allowing operators to identify problematic URLs or sites.

## Reflections and Future Work

Building this scraper reinforced the importance of respecting rate limits and server resources, not just for ethical reasons, but because aggressive scraping invariably leads to IP blocks and degraded data quality. Future enhancements could include distributed scraping across multiple machines (coordinated through Kafka), JavaScript rendering support, and more sophisticated content extraction using machine learning models trained to identify main article content.

## 2.3 Orchestrating Services and Visualizing Data with OpenSearch

Modern data systems comprise numerous interconnected services, each specialized for a particular task. Orchestrating these services, ensuring they start in the correct order, communicate reliably, and remain healthy, is a discipline unto itself. Through this project, I gained hands-on experience with Docker Compose orchestration and OpenSearch as both a search engine and visualization platform.

### Container Orchestration with Docker Compose

Docker Compose transforms complex multi-service deployments into declarative YAML configurations. Our pipeline includes OpenSearch (with k-NN plugin), OpenSearch Dashboards, Kafka (with Zookeeper), and our custom Python services. Without containerization, installing and configuring these services would require hours of manual work and vary across different operating systems. With Compose, a single `docker-compose up` command brings the entire stack online.

Service dependencies required careful handling. OpenSearch must be healthy before our Python services attempt to create indices or index documents. Kafka requires Zookeeper. These dependencies are expressed through `depends_on` declarations with health checks, ensuring services start in the correct order. I implemented custom health check scripts that verify not just that a process is running, but that it's actually ready to accept connections.

Networking between containers uses Docker's bridge network, with services addressing each other by name (`opensearch`, `kafka`, etc.). This DNS-based discovery eliminates hardcoded IP addresses and adapts automatically as containers restart. Environment variables passed through Compose configure each service's connection parameters, maintaining a single source of truth for configuration.

### OpenSearch: More Than Full-Text Search

OpenSearch, an open-source fork of Elasticsearch, provides the search backbone for our pipeline. While traditionally known for full-text search, OpenSearch's k-NN plugin enables vector similarity search, essential for semantic search applications. Understanding and configuring k-NN required studying approximate nearest neighbor algorithms, particularly HNSW (Hierarchical Navigable Small World) graphs.

Index mapping design proved critical for search quality. Our index defines a vector field with 384 dimensions (matching our embedding model), uses HNSW for the k-NN engine, and specifies parameters like `m` (number of neighbors per graph node) and `ef_construction` (search width during indexing). These parameters trade indexing speed against query performance, higher values improve recall but increase memory and CPU usage.

The OpenSearch client wrapper I developed abstracts low-level HTTP operations into a clean Python interface. Bulk indexing uses OpenSearch's helpers for efficient batch operations, critical when indexing thousands of documents. The wrapper handles connection management, retries with exponential backoff, and proper error classification (distinguishing transient failures from permanent errors).

## Building the Search Interface

Streamlit provided a rapid development path for our search frontend. The interface offers three search modes: semantic (pure vector search), text (traditional BM25), and hybrid (combining both with configurable weights). Users can adjust parameters like the number of results and the semantic/text balance for hybrid search.

Results rendering includes relevance scores, source URLs, and snippet previews. A key challenge was presenting vector similarity scores meaningfully to users, raw cosine similarity values lack intuitive interpretation. We normalize and scale scores to percentages, giving users a sense of match quality.

## OpenSearch Dashboards for Monitoring

OpenSearch Dashboards (the visualization companion to OpenSearch) provides powerful capabilities for monitoring pipeline health. I created dashboards tracking document ingest rates, index sizes, query latencies, and error rates. These visualizations transformed raw metrics into actionable insights, we quickly spotted anomalies like sudden drops in ingest rate or spikes in query latency.

The Discover feature allows ad-hoc exploration of indexed documents, invaluable during debugging. When semantic search results seemed off, I could examine individual documents to verify embeddings were generated correctly and text was properly chunked.

## Operational Lessons

Running containerized services taught valuable operational lessons. Resource limits matter, OpenSearch is memory-hungry, and without explicit limits, it can consume all available RAM. Log aggregation across multiple containers requires strategy; we route logs to stdout and use Docker's logging drivers for centralized collection. Persistent volumes ensure data survives container restarts, but volume management requires planning for growth.

## 2.4 Implementing a Semantic Processing Pipeline for Text Embeddings

Semantic search represents a paradigm shift from keyword matching to meaning understanding. At the heart of this transformation lies the embedding generation process, converting text into dense vector representations where semantic similarity corresponds to geometric proximity. Building the text processing pipeline for our project provided deep insights into natural language processing, chunking strategies, and the practical challenges of embedding generation at scale.

### Understanding Embeddings

Traditional text representations like TF-IDF (Term Frequency-Inverse Document Frequency) map documents to sparse vectors where each dimension corresponds to a word in the vocabulary. While effective for keyword search, these representations fail to capture synonymy (different words with similar meanings) or polysemy (same word with different meanings). A TF-IDF vector for "automobile" is orthogonal to one for "car" despite their semantic equivalence.

Dense embeddings from neural language models address this limitation. Models like BERT and its derivatives learn contextual representations during training on massive text corpora. The sentence-transformers library fine-tunes these models specifically for sentence similarity tasks, producing embeddings where semantically similar texts cluster together. For our pipeline, I selected `all-MiniLM-L6-v2`, a model offering 384-dimensional embeddings with an excellent balance of quality and speed.

### The Chunking Challenge

Embedding models have input length limitations, and even without hard limits, longer texts produce less focused embeddings. A 5000-word article embedded as a single vector loses granularity, queries about specific sections return the entire document or nothing. Chunking addresses this by splitting documents into smaller segments, each independently embedded.

Our chunker implements a sliding window approach with configurable chunk size (default 500 tokens) and overlap (default 50 tokens). The overlap ensures concepts spanning chunk boundaries remain searchable, without it, a query might match neither chunk despite the answer existing in the original text. Token counting approximates using word count multiplied by 1.3 (a heuristic for English text tokenization), avoiding the overhead of running the actual tokenizer during chunking.

Respecting sentence boundaries improves chunk quality. Breaking mid-sentence creates fragments lacking context. The chunker identifies sentence boundaries using regex patterns and adjusts chunk boundaries to align with complete sentences when possible, even if this produces slightly smaller or larger chunks than the target size.

### Batch Processing for Efficiency

Embedding generation is computationally intensive, especially on CPU. Processing documents one at a time incurs Python loop overhead and underutilizes hardware parallelism. The embedding generator implements batch processing, accumulating chunks before en-

coding them together. The sentence-transformers library efficiently batches these computations, leveraging NumPy's vectorized operations and optional GPU acceleration.

Memory management required attention when processing large document sets. Embeddings are 384-dimensional float32 vectors, consuming about 1.5KB each. Processing millions of chunks simultaneously would exhaust memory. The pipeline uses generators to stream chunks, processing and indexing batches incrementally rather than accumulating everything in memory.

## Normalization and Similarity

Embedding vectors can be used with various similarity metrics: cosine similarity, Euclidean distance, and dot product. Cosine similarity is standard for semantic search, measuring the angle between vectors regardless of magnitude. To enable efficient dot product computation (equivalent to cosine similarity for unit vectors), the embedding generator normalizes vectors to unit length. This normalization happens once during embedding generation, avoiding repeated computation during search.

## Integration and Testing

The embedding module integrates with both the pipeline runner (for batch indexing) and the search frontend (for query embedding). Query embedding must use the identical model and normalization as document embedding, any discrepancy breaks semantic similarity. I implemented caching for the model to avoid reloading for each query and added validation ensuring embedding dimensions match the index configuration.

Testing semantic search quality presents unique challenges. Unlike keyword search with clear precision/recall metrics, semantic similarity is subjective. I assembled test query sets with expected relevant documents, measuring whether correct documents appeared in top results. This qualitative testing identified edge cases where chunking disrupted context or embedding model limitations caused unexpected misses.

## Reflections on Neural Search

Building this pipeline deepened my appreciation for the complexity underlying "simple" semantic search. The interplay between chunking strategy, embedding model choice, and similarity computation creates a vast parameter space affecting search quality. Future work could explore re-ranking retrieved results with cross-encoder models, combining multiple embedding models for improved coverage, or fine-tuning embeddings on domain-specific data.

### 3 Implementation Description

The implementation follows a modular architecture with clear separation of concerns across five major components: configuration management, data acquisition, message queuing, semantic processing, and search/visualization.

**Configuration Management** (`src/config.py`) centralizes all settings using Python dataclasses with environment variable support. This includes Kafka connection parameters (`KAFKA_BOOTSTRAP_SERVERS`, `KAFKA_TOPIC_RAW_DATA`), OpenSearch settings (`OPENSEARCH_HOST`, `OPENSEARCH_PORT`, `OPENSEARCH_INDEX`), embedding model configuration (`EMBEDDING_MODEL`, `EMBEDDING_DIMENSION`), and chunking parameters (`CHUNK_SIZE`, `CHUNK_OVERLAP`). The configuration module loads environment variables from a `.env` file via `python-dotenv`, enabling environment-specific settings without code changes.

**Data Acquisition** (`src/scrapper/`) implements a `WebScraper` class that fetches HTML content, extracts clean text, and generates structured `ScrapedDocument` objects. The scraper respects rate limits through configurable delays, rotates user agents via the `fake-useragent` library, and handles network failures with retry logic using the `tenacity` library. The companion `ContentExtractor` class employs `BeautifulSoup` for HTML parsing, removing non-content elements (scripts, styles, navigation) and extracting the main article text with proper encoding handling.

**Message Queuing** (`src/kafka/`) provides `KafkaProducerWrapper` and `KafkaConsumerWrapper` classes abstracting `kafka-python` operations. The producer implements reliable delivery with configurable acknowledgment levels, gzip compression, and batch accumulation. The consumer supports manual offset commits for at-least-once processing guarantees, graceful shutdown handling via signal handlers, and statistics tracking for monitoring. Both wrappers include automatic reconnection logic and comprehensive error handling.

**Semantic Processing** (`src/processing/`) contains the `TextChunker` and `EmbeddingGenerator` classes. The chunker implements sliding-window segmentation with configurable token overlap and sentence boundary awareness. Each chunk preserves metadata linking it to its source document. The embedding generator wraps sentence-transformers, supporting multiple models (defaulting to `all-MiniLM-L6-v2`), batch processing for efficiency, and L2 normalization for consistent similarity computation. Both components integrate seamlessly, with `EmbeddedChunk` objects combining chunk text with its vector representation.

**Storage and Search** (`src/opensearch/`) implements an `OpenSearchClient` handling connection management, index creation with k-NN mappings, bulk document indexing, and multi-modal search. The index mapping configures HNSW parameters for approximate nearest neighbor search. The client supports three search modes: pure semantic (k-NN), pure text (BM25), and hybrid (score-weighted combination). Bulk indexing uses OpenSearch helpers for efficient batch operations.

**Pipeline Orchestration** (`src/pipeline/`) ties all components together through the `DataPipeline` and `PipelineRunner` classes. The pipeline can operate in streaming mode (consuming from Kafka) or batch mode (processing files directly). The runner coordinates the workflow: scraping URLs, publishing to Kafka, consuming messages, chunking text, generating embeddings, and indexing to OpenSearch.

**Frontend** (`src/frontend/app.py`) provides a Streamlit-based web interface with search type selection (semantic/text/hybrid), adjustable parameters (result count, hybrid weights), and formatted result display with relevance scores and source URLs.

**Containerization** (`docker-compose.yml`) defines services for OpenSearch (with k-NN plugin), OpenSearch Dashboards, Kafka, Zookeeper, and the Python application. Health checks ensure proper startup ordering, and persistent volumes maintain data across restarts.

## 4 Conclusions

This project successfully demonstrates the feasibility of building a production-ready semantic search pipeline using open-source technologies. By combining Apache Kafka for resilient message streaming, sentence-transformers for embedding generation, and OpenSearch for hybrid vector-lexical search, the system achieves a flexible architecture capable of handling web-scale data ingestion while providing semantically-aware search capabilities. The modular design allows individual components to be upgraded or replaced independently, and the Docker Compose orchestration ensures reproducible deployments. Key lessons learned include the importance of proper chunking strategies for embedding quality, the need for resilient error handling in distributed systems, and the value of hybrid search combining semantic understanding with traditional keyword matching. Future enhancements could include support for real-time index updates, cross-encoder re-ranking for improved precision, and expansion to multilingual content through appropriate embedding models.

## References

- [1] Kreps, J., Narkhede, N., & Rao, J. (2011). *Kafka: A Distributed Messaging System for Log Processing*. Proceedings of the NetDB Workshop.
- [2] Apache Kafka Documentation. (2024). *Apache Kafka®*. Apache Software Foundation. <https://kafka.apache.org/documentation/>
- [3] Mitchell, R. (2018). *Web Scraping with Python: Collecting More Data from the Modern Web* (2nd ed.). O'Reilly Media.
- [4] Richardson, L. (2024). *Beautiful Soup Documentation*. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- [5] OpenSearch Project. (2024). *OpenSearch Documentation*. <https://opensearch.org/docs/latest/>
- [6] OpenSearch Project. (2024). *k-NN Plugin*. <https://opensearch.org/docs/latest/search-plugins/knn/index/>
- [7] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).
- [8] HuggingFace. (2024). *Sentence Transformers Documentation*. <https://www.sbert.net/>